

# **Manual de usuario de Github**

---

## Manual de usuario de Github

---

---

# Table of Contents

|                                                    |   |
|----------------------------------------------------|---|
| I. Guia de principiantes .....                     | 1 |
| 1. Introducción .....                              | 3 |
| ¿Qué es GitHub? .....                              | 3 |
| 2. Creación de cuenta y configuración básica ..... | 4 |
| Registro y Configuración inicial .....             | 4 |
| 3. Conceptos básicos de GitHub .....               | 5 |
| Repositorios .....                                 | 5 |
| Commits .....                                      | 5 |
| Issues .....                                       | 6 |
| 4. Conclusiones .....                              | 7 |

---

# **Part I. Guia de principiantes**

---

---

## Table of Contents

|                                                    |   |
|----------------------------------------------------|---|
| 1. Introducción .....                              | 3 |
| ¿Qué es GitHub? .....                              | 3 |
| 2. Creación de cuenta y configuración básica ..... | 4 |
| Registro y Configuración inicial .....             | 4 |
| 3. Conceptos básicos de GitHub .....               | 5 |
| Repositorios .....                                 | 5 |
| Commits .....                                      | 5 |
| Issues .....                                       | 6 |
| 4. Conclusiones .....                              | 7 |

---

# Chapter 1. Introducción

## ¿Qué es GitHub?

GitHub es una plataforma en línea que permite alojar y gestionar proyectos de desarrollo de software usando el sistema de control de versiones Git. Es una comunidad global donde desarrolladores, empresas y organizaciones colaboran en proyectos compartidos, realizan revisiones de código, gestionan tareas y documentan su trabajo. La plataforma facilita la colaboración en equipo y la gestión eficiente de cambios en archivos, permitiendo hacer seguimiento histórico, revertir versiones anteriores, y fusionar aportaciones.

*Imagina un proyecto de código abierto para un app móvil. Varios desarrolladores en distintas partes del mundo trabajan en diferentes funcionalidades, haciendo commits y creando issues para bugs o nuevas ideas. GitHub centraliza todo, permitiendo que todos contribuyan de manera ordenada y transparente.*

---

# Chapter 2. Creación de cuenta y configuración básica

## Registro y Configuración inicial

1. **Crear una cuenta:** Ir a <https://github.com>, rellenar el formulario con usuario, email y contraseña. Se recomienda usar una contraseña segura y activar la verificación en dos pasos para mayor seguridad.
2. **Verificación del email:** Recibirás un correo para verificar tu cuenta, así que deberás entrar a tu correo y presionar el botón donde ponga "**Verificar**".
3. **Configuración del perfil:** Subir una foto, añadir una breve bio, enlaces a tu blog o portafolio profesional. Esto mejora tu presencia y credibilidad en la comunidad.
4. **(Opcional) Configuración de SSH/HTTPS:** Para sincronizar tus repositorios locales con GitHub, configura claves SSH o token de acceso personal. Esto evita introducir tu usuario y contraseña repetidamente.

---

# Chapter 3. Conceptos básicos de GitHub

## Repositorios

- ¿Qué son?

Un repositorio de GitHub es el espacio central donde un proyecto almacena todos sus archivos y el historial de cambios a lo largo del tiempo. Dentro de un repositorio puedes tener cualquier tipo de archivos: código fuente, imágenes, documentos, configuración e incluso manuales. Además, los repositorios guardan metadatos importantes como quién ha hecho cada cambio, cuándo y por qué. GitHub proporciona numerosos servicios integrados en los repositorios, como el seguimiento de problemas, las solicitudes de cambio, la wiki del proyecto y las opciones de seguridad avanzadas.

- Cómo crearlos:

Puedes crear un repositorio en la web de GitHub haciendo clic en "New Repository". Se solicita un nombre y puedes añadir una descripción. Es importante decidir si tu repositorio será público, para que cualquier usuario lo vea, o privado, para mantenerlo restringido. Es muy recomendable incluir un archivo README, que explica el propósito del proyecto, y un archivo .gitignore, que indica qué archivos no se deben subir. Una vez creado, puedes clonar el repositorio en tu equipo local usando Git y así trabajar de forma remota y sincronizada.

*Por ejemplo, al crear el repositorio "MiProyectoWeb", puedes alojar el código para una página web personal. El repositorio contendrá el archivo index.html, imágenes del sitio, documentación en el README, y una lista de problemas a resolver en la pestaña Issues. Si varios usuarios contribuyen, cada cambio queda registrado y referenciado con el usuario y la fecha correspondientes.*

## Commits

- ¿Qué son?

Un commit es una instantánea del estado de los archivos de un repositorio en un momento determinado. Cada vez que haces cambios en tu proyecto, puedes guardar esos cambios en un commit. De esta forma, GitHub mantiene un historial completo de lo que ha cambiado y quién realizó esa modificación. Los commits pueden tener mensajes descriptivos que ayudan al equipo a entender el propósito de cada conjunto de cambios, lo que facilita la revisión y el mantenimiento del proyecto, incluso después de años de desarrollo.

- Como hacer un commit

1. Seleccionar los archivos que quieres agregar al commit
2. Escribe un mensaje explicativo y guarda tu commit. Los mensajes claros y específicos mejoran la colaboración, por ejemplo: "Arreglo del error al guardar usuario", "Agrego pantalla de estadísticas".

Para hacer un commit, normalmente usas comandos de Git o la interfaz visual de GitHub Desktop o VSCode. El proceso básico es: Es importante hacer commits frecuentes y de pequeños cambios para mantener el historial organizado. En GitHub, los commits pueden ser revisados por otros usuarios mediante pull requests, antes de llegar al código principal.

*Supón que modificas el color del fondo en tu aplicación y le cambia el tono a otro. El commit debe reflejar este cambio con un mensaje como: "Cambio de color del fondo".*



# Issues

- **¿Qué son?**

Las Issues en GitHub son el sistema que permite identificar, registrar y gestionar tareas, ideas o problemas relacionados con el proyecto. Son útiles tanto para grandes equipos como para trabajos individuales, ya que ofrecen trazabilidad, comunicación y documentación de los retos del proyecto. Dentro de una issue puedes describir el problema, asignar responsables, definir etiquetas (“bug”, “enhancement”, “question”), establecer prioridades e incluso relacionar issues con pull requests, automatizando su cierre si la solución está en el código. Issues también sirven para discutir propuestas de cambios, preguntar dudas importantes y dividir el trabajo en subtareas.

- **Cómo usarlos**

1. Accede a la pestaña Issues de tu repositorio y pulsa “**New Issue**”.
2. Redacta una descripción precisa, añade pasos para reproducir el problema si es un bug, adjunta capturas o códigos de ejemplo, y etiqueta la issue según su tipo.

A medida que avanzas, actualiza el estado de la issue con comentarios y, cuando la tarea esté completa, puedes cerrarla. Si la issue representa una tarea grande, puedes descomponerla en sub-issues o listas de tareas, logrando una mejor organización.

*En el desarrollo de una plataforma de cursos online, puedes crear una issue llamada “Error al cargar video en navegadores móviles”. Esta issue tendrá una descripción del problema, pasos para reproducirlo, evidencia visual (captura de pantalla del error) y estará asignada a un miembro del equipo.*

---

## Chapter 4. Conclusiones

GitHub es esencial para colaborar en proyectos de software, permitiendo gestionar versiones, revisar cambios y coordinar tareas. Crear una cuenta, entender los conceptos de repos, commits e issues, y seguir buenas prácticas, son los pasos fundamentales para empezar a contribuir o gestionar tus propios proyectos.